Lab Assignment 1: Convolutional Neural Network (CNN)

Name	Krushna Prashant Warule	**Dept**	Computer
Roll No	_____	**Div**	B
Subject	Advance Data Science	**Assignment No**	1
GitHub	https://github.com/krushnawarule/Advance_Data_Science_Lab_SCOE/tree/main/Assignment_1_CNN		

1. Aim

To study and implement a Convolutional Neural Network (CNN) for image classification and understand how convolution, activation, pooling, and fully connected layers work together to extract features and classify images.

2. Theory

Convolution slides small learnable filters over the image to detect local patterns such as edges and strokes. **ReLU** adds non-linearity by zeroing negative values. **Max pooling** keeps the strongest response in each 2x2 region, which shrinks the feature map and makes the network tolerant to small shifts. The pooled maps are **flattened** and passed to **fully connected layers**, and a **softmax** layer outputs a probability for each class. All weights are learned with backpropagation and the Adam optimizer.

Architecture used: Input 8x8 -> Conv(8 filters, 3x3) -> ReLU -> MaxPool(2x2) -> Flatten(72) -> Dense(32)+ReLU -> Dense(10)+Softmax. The network is implemented from scratch in NumPy (forward and backward pass written manually) because TensorFlow/Keras was not available in the working environment.

3. Dataset

Handwritten Digits dataset (scikit-learn): 1797 grayscale images of size 8x8, 10 classes (digits 0-9). Saved as `dataset.csv` (64 pixel columns + label). Split: 80% train (1437), 20% test (360).

4. Code (core part; full code in code.py)

```
class SmallCNN:
    """Conv(3x3,8) -> ReLU -> MaxPool(2x2) -> Flatten -> Dense(32)+ReLU -> Dense(10)+Softmax"""
    def __init__(self, nf=8, nclass=10):
        r = np.random.randn; self.nf = nf
        self.p = {"K": r(nf, 3, 3) * 0.3, "kb": np.zeros(nf),
                  "W1": r(nf * 9, 32) * np.sqrt(2 / (nf * 9)), "b1": np.zeros(32),
                  "W2": r(32, nclass) * np.sqrt(2 / 32), "b2": np.zeros(nclass)}

    def forward(self, X):
        p = self.p; N = len(X)
        win = swv(X, (3, 3), axis=(1, 2))                                   # sliding 3x3 windows
        conv = np.einsum("nijkl,fkl->nfij", win, p["K"]) + p["kb"][None, :, None, None]
        act = np.maximum(conv, 0)                                           # ReLU activation
        blk = act.reshape(N, self.nf, 3, 2, 3, 2)
        pool = blk.max(axis=(3, 5))                                         # 2x2 max pooling
        flat = pool.reshape(N, -1)                                          # flatten
        pre = flat @ p["W1"] + p["b1"]; h = np.maximum(pre, 0)              # fully connected + ReLU
        z = h @ p["W2"] + p["b2"]
        e = np.exp(z - z.max(1, keepdims=True))
        return e / e.sum(1, keepdims=True), (win, conv, act, blk, pool, flat, pre, h)

    def backward(self, y, P, c):
        p = self.p; win, conv, act, blk, pool, flat, pre, h = c; N = len(y)
        d = P.copy(); d[np.arange(N), y] -= 1; d /= N                       # softmax + cross-entropy
        g = {"W2": h.T @ d, "b2": d.sum(0)}
        dh = (d @ p["W2"].T) * (pre > 0)
        g["W1"] = flat.T @ dh; g["b1"] = dh.sum(0)
        dpool = (dh @ p["W1"].T).reshape(pool.shape)
        mask = (blk == pool[:, :, :, None, :, None]).astype(float)
        mask /= mask.sum(axis=(3, 5), keepdims=True)
        dact = (mask * dpool[:, :, :, None, :, None]).reshape(N, self.nf, 6, 6)
        dconv = dact * (conv > 0)
        g["K"] = np.einsum("nijkl,nfij->fkl", win, dconv); g["kb"] = dconv.sum((0, 2, 3))
        return g

    def fit(self, X, y, epochs=30, bs=32, lr=0.01, frozen=(), val=None, log=True):
        st = {"t": 0, "m": {k: 0 * v for k, v in self.p.items()}, "v": {k: 0 * v for k, v in self.p.items()}}
        hist = {"loss": [], "acc": [], "val": []}
        for ep in range(1, epochs + 1):
            order = np.random.permutation(len(X))
            for i in range(0, len(X), bs):
                b = order[i:i + bs]; P, c = self.forward(X[b]); g = self.backward(y[b], P, c)
                st["t"] += 1
                for k in self.p:
```

```
                if k in frozen: continue                                # frozen layers are not updated
                st["m"][k] = 0.9 * st["m"][k] + 0.1 * g[k]
                st["v"][k] = 0.999 * st["v"][k] + 0.001 * g[k] ** 2
                mh = st["m"][k] / (1 - 0.9 ** st["t"]); vh = st["v"][k] / (1 - 0.999 ** st["t"])
                self.p[k] -= lr * mh / (np.sqrt(vh) + 1e-8)
            P, _ = self.forward(X)
            hist["loss"].append(-np.log(P[np.arange(len(y)), y] + 1e-9).mean())
            hist["acc"].append((P.argmax(1) == y).mean())
            if val is not None: hist["val"].append((self.predict(val[0]) == val[1]).mean())
            if log and (ep % 5 == 0 or ep == 1):
                print(f"Epoch {ep:2d}/{epochs}  loss={hist['loss'][-1]:.4f}  train_acc={hist['acc'][-1]:.4f}"
                      + (f"  test_acc={hist['val'][-1]:.4f}" if val is not None else ""))
        return hist

    def predict(self, X): return self.forward(X)[0].argmax(1)
```

5. Output

Console output (output.txt):

```
Dataset: 1797 images of size 8x8, 10 classes | train=1437 test=360
Trainable parameters: 2746
Epoch  1/30  loss=0.6372  train_acc=0.8038  test_acc=0.7833
Epoch  5/30  loss=0.0979  train_acc=0.9715  test_acc=0.9333
Epoch 10/30  loss=0.0460  train_acc=0.9868  test_acc=0.9583
Epoch 15/30  loss=0.0128  train_acc=0.9972  test_acc=0.9806
Epoch 20/30  loss=0.0138  train_acc=0.9979  test_acc=0.9667
Epoch 25/30  loss=0.0033  train_acc=1.0000  test_acc=0.9667
Epoch 30/30  loss=0.0024  train_acc=1.0000  test_acc=0.9667

Final Test Accuracy: 96.67%

              precision    recall  f1-score   support

           0      1.000     0.977     0.989        44
           1      0.944     0.895     0.919        38
           2      0.944     1.000     0.971        34
           3      0.973     1.000     0.986        36
           4      0.953     1.000     0.976        41
           5      0.971     1.000     0.985        33
           6      0.968     0.968     0.968        31
           7      1.000     0.967     0.983        30
           8      0.947     0.923     0.935        39
           9      0.970     0.941     0.955        34

    accuracy                          0.967       360
   macro avg      0.967     0.967     0.967       360
weighted avg      0.967     0.967     0.966       360
```

Output images:

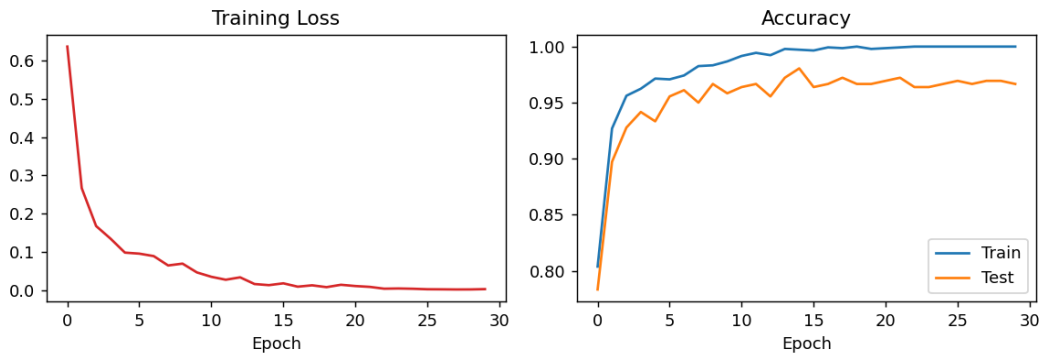


Fig 1: Training loss and accuracy per epoch

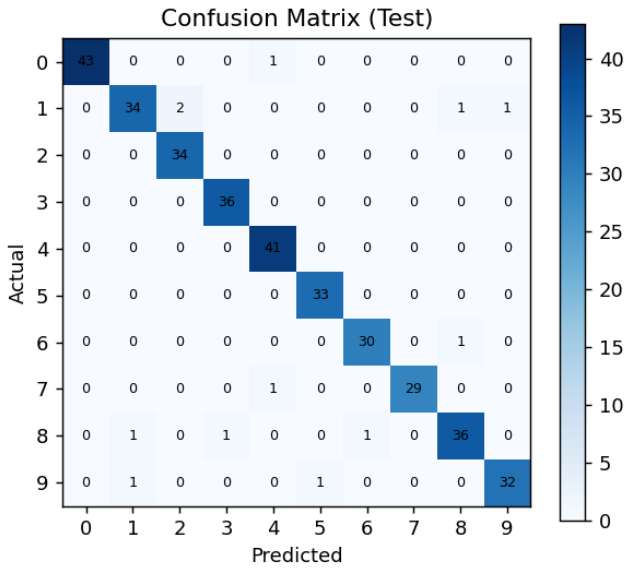


Fig 2: Confusion matrix on the test set

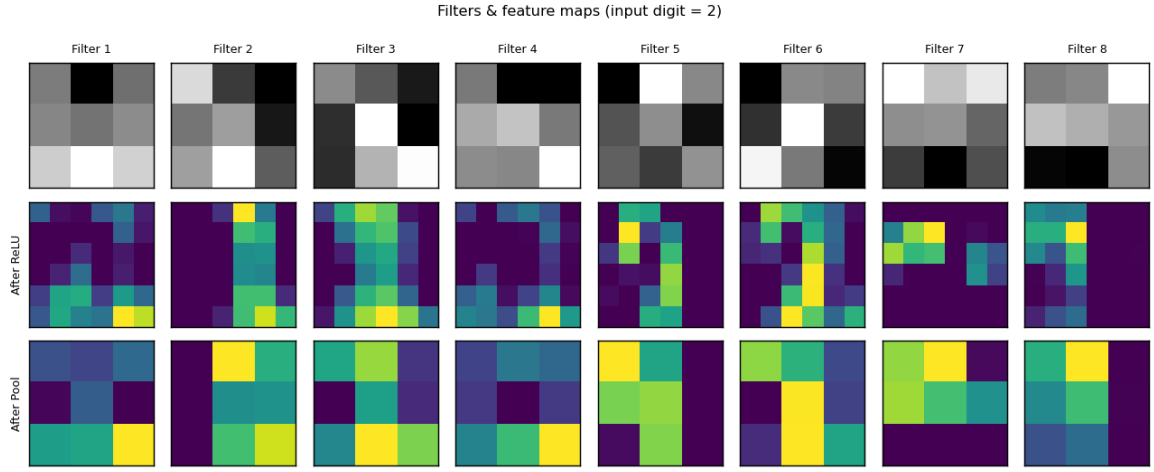


Fig 3: Learned 3x3 filters, feature maps after ReLU and after pooling

6. Conclusion

The CNN reached a test accuracy of **96.67%** with only 2,746 trainable parameters. The training loss fell steadily while accuracy rose, showing that the convolution filters learned useful stroke-like features and that pooling plus fully connected layers converted them into correct class decisions. The confusion matrix shows most errors occur between visually similar digits (for example 1 and 8). Training accuracy reached 100% while test accuracy stayed around 96-98%, a small sign of overfitting that data augmentation or dropout could reduce.

7. References

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, 'Gradient-based learning applied to document recognition', Proceedings of the IEEE, 1998.

[2] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016 (Chapter 9: Convolutional Networks).

[3] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python', JMLR 12, 2011 (Digits dataset).

[4] D. Kingma, J. Ba, 'Adam: A Method for Stochastic Optimization', ICLR 2015.